Lecture 5

Methods

Organization! Structure!

- With loops and arrays we can program every computation that can be programmed at all
- Beyond that, we need:
  - good organization of our code
  - organized access to code by others, used by us
- note: most of what your program does is written by others

Talking about organization, what day is it today?



A. King's Day

B. Mad Tuesday

C. Prince's Day

D. Fat Tuesday

Answer

Control structuring - methods

- A method is a sequence of statements with a name
- Calling (aka invoking) the name leads to execution of the sequence of statements
- After execution, control returns to the place after the call

Defining and using a method

Define a class

define the name of the method

define the body

call the method

again

and again

call the method chorus

```
class Crowd {

   void shout() {
      System.out.println("Hij is een hondelul");
   }

   void chorus() {
      shout();
      shout();
      shout();
   }

   public static void main(String[] a)
      new Crowd ().chorus();
   }
}
```

output

```
Hij is een hondelul
Hij is een hondelul
Hij is een hondelul
```

method - syntax

1. declaration and definition:

```
void <name> () {
    <statements>                    "method body"
}
```

2. use (call):

```
<name>();
```

- "void" is only used at the definition
- word void will be explained later


INTO THE
VOID

Why use methods?

- saves time with recurring tasks
- makes your program more readable
- gives a better organization of your program:
  - subdivision of larger tasks
  - group tasks that belong together
- makes your program more flexible, i.e., reduces dependencies between parts
  - more later
- makes (parts of) your code reusable

Recurring tasks

```
class Emoticon {
    void smile() {
      System.out.println( ":-)" );
    }

    void sad() {
      System.out.println( ":-(" );
    }

    void angry() {
      System.out.println( ">:-|" );
    }

    void message() {
     System.out.println("Hi, Irma");
     smile();
     System.out.println("Yesterday I saw Annechien");
     angry();
     angry();
     angry();
    }
}
```

Output:

```
Hi, Irma
:-)
Yesterday I saw Annechien
ss
>:-|
>:-|
```

new Emoticon().message; in main

Reusability: parameters

```
class Emoticon {
   void smile() {
     System.out.println( ":-)" );
   }

   void angry(int n) {
     for (int i=0; i<n; i++) {
        System.out.println( ">:-|" );
     }
   }

   void demoticon {
      System.out.println("Hi, Plien");
      smile();
      System.out.println("Gister zag ik Annechien");
      angry(3);
   }
}
```

Output:

```
Hi, Plien
:-)
Gister zag ik Annechien
>:-|
>:-|
>:-|
```

method as organizer of statements

- a method groups one or more **statements**
- gives it a name
- hides local variables and implementation
  - local variables are 'hidden', i.e., not visible or changeable by code outside the method
  - same for how an effect is achieved (=implementation)

Hiding information

- Hiding information reduces the dependencies between parts of the program
- decreases complexity
- increases flexibility
  - changes in hidden things do not affect other parts

method as organizer of expressions

- expressions are pieces of Java that result in a value:
  - `x`
  - `x + y`
  - `x / y - 2 * (x + y)`
  - `0 <= x && x < n`
  - `Math.sqrt(b * b - 4 * a * c)`
- can we group and name **expressions** and hide internals?
- Yes! ⇒ **functions**

functions

- known from mathematics: square root, formulas, functions
- mathematics: $f(x) = x^2 + 4x + 7$

```
class Example {
   double f(double x) {
      return x*x + 4*x + 7;
   }
   …
```

parameter

return type

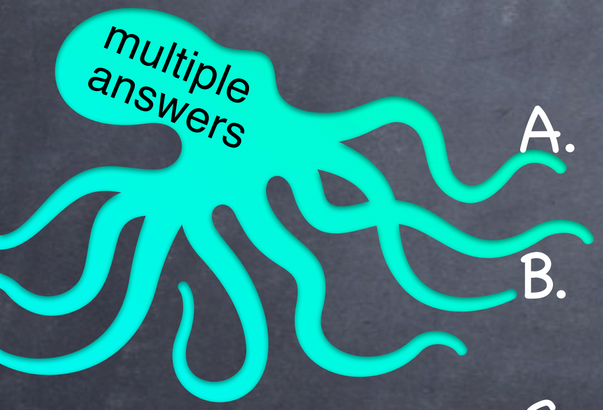
return value

- examples of use (inside a method in Example):

```
double a, b, q;   q = 3.141;
a = f(10);
b = f(5) + f(q/2);
System.out.println("value for 3: " + f(3));
```

- parameter x is hidden to code outside method

Question on hiding





- What are effects of hiding things (variables, implementation)

A. It improves handling of changes in the code

B. It improves efficiency

C. It decreases dependencies between code parts

D. It makes it harder to understand the code

Answer

Notes on functions

- functions are methods with return value
- typical purpose of functions is "mathematical" calculations etc.
- side-effects preferably not in functions:
  - no change of instance variables
  - no input or output
- pure functions: functions that do not change or use the state

Further note

- When a method does not return a value, its return type is denoted as void
- Do not call a method "a void"
- Unless you call a cow "a milk" and a professor "a blahblah"

What does the box do (1)?




```
class Printer {
  void hek(int n) {
    String s = "";
    for (int i=0; i<n; i++) {
      s = s+"#";
    }
    System.out.println(s);
  }

  void box() {
    hek(1);
    hek(2);
    hek(3);
    hek(4);
  }

  public static void main(String[] a) {
     new Printer().box(); }
}
```

A. ##########

B.
```
#
##
###
####
```

C.
```
#
#
#
#
```

D.
```
####
###
##
#
```

What does this box do (2)?

```
class Printer {
  void hek(int n) {
    String s = "";
    for (int i=0; i<n; i++) {
      s = s + “#";
    }
    System.out.print(s);
  }

  void nl() {
    System.out.println();
  }
```

```
void box() {
    hek(8); nl();
    hek(2); nl();
    hek(6); nl();
    hek(2); nl();
    hek(2); nl();
    hek(2); nl();

  }
}
```

A.

B.

C.

D.

E.

F.

Method types

I. simple

```
void <name>() {
  <body>
}
```

all methods are declared at class level, not inside other methods

II. parameters

```
void <name>(<type> <var>,..., <type> <var>) {
  <body>
}
```

rare; usually

III. return

```
<type> <name>() {
  <body>
  return <val>;
}
```

IV. combination of II and III

Method examples

I. simple

```
void hello() {
  System.out.println("Hellooooo!");
}
```

II. parameters

```
void repeat(String s, int n) {
  for (int i=0;i<n;i++){ System.out.println(s); }
}
```

III. return

```
double pi() {
  return 3.14159265359;
}
```

IV. combination of II and III

```
int max (int a, int b) {
  if (a>b){return a;}else{return b;}
}
```

Name for function



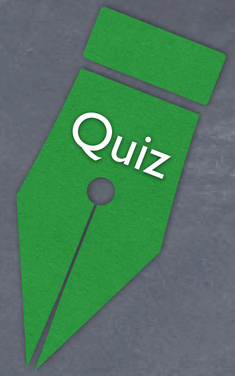
- consider the expression `b*b - 4*a*c`
- if we make this into a function, what would be a good name for it?

Quadratic equations



- $ax^2 + bx + c = 0$
- solutions: $x_{1,2} = \frac{-b \pm \sqrt{b^2 - 4ac}}{2a}$
- D > 0 ⇒ 2 solutions
- D = 0 ⇒ 1 solution
- D < 0 ⇒ 0 solutions

Function question 2



- consider again the expression `b*b - 4*a*c`
- suppose its name will be discriminant
  what does the declaration look like?

A. `void discriminant()`

B. `void discriminant(double a, double b, double c)`

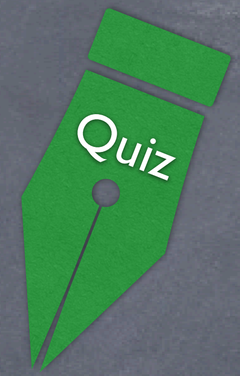
C. `double discriminant()`

D. `double discriminant(double a, double b, double c)`

Answer

Applying (calling) discriminant

```
void happy() {
  System.out.println("there
    are two solutions!");
}
```



- How to call the function discriminant?

A. `if (discriminant > 0) { happy(); }`

B.
```
double D = discriminant();
if (D > 0) { happy(); }
```

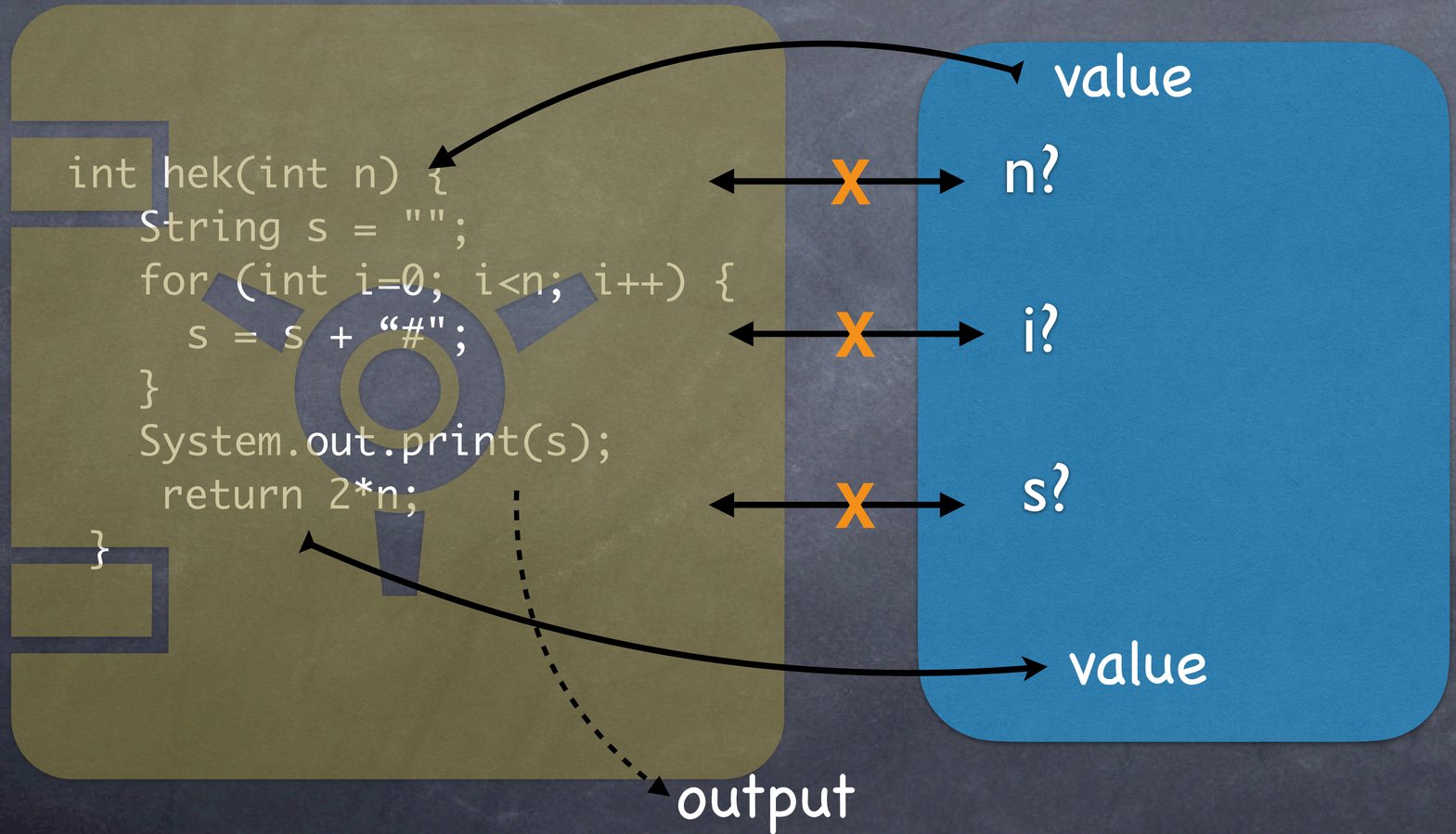
C. `if (discriminant(c, 2, b) > 0) { happy(); }`

D. `if (discriminant(a, b) > 0) { happy(); }`

Answer

A little bit about scope

- variables and parameters of a method are not visible from the outside (other methods)



we say that the scope of parameter n and of the local variables i and s is the method body

Answers to quizzes

- slide 15 (Hiding): A and C
- slide (Box 1): B
- slide (Box 2): F
- slide (function name): determinant or discriminant
  D is seen in school books, but a bit unspecific for a program
- slide (function 2): D
- slide (calling): C

Methods as building blocks

- Methods are the building blocks, the units, of software
- Protected inner: local variables are not accessible from outside
- Can be tested separately: unit testing

- scope makes functions closed boxes
- reduces dependencies and this is a good thing

```
int sum(int[] a) {
    int s = 0;
    for (int i = 0; i < a.length; i++) {
      s += a[i];
    }
    return s;
  }
```

```
int sum(int[] b) {
    int t = 0;
    for (int j = 0; j < b.length; i++) {
      t += b[i];
    }
    return t;
  }
```

```
int sum(int[] row) {
    int t = 0;
    for (int j = 0; j < row.length; i++) {
      if (row[i] != 0) {
        t += row[i];
      }
    }
    return t;
  }
```

```
int sum(int[] a) {
    int s = 0;
    for (int elt : a) {
      s += elt;
    }
    return s;
  }
```

```
int sum(int[] a) {
    int s = 0;
    int i = a.length;
    while (i > 0) {
      i = i - 1;
      s += a[i];
    }
    return s;
  }
```

- all are the same as regards the rest of the code
- no dependencies on names or inner workings